TwinPrompt: Dual-View Time-Series Prompting for Few-Shot Generative Classification

Shizhuo Deng, *Member, IEEE*, Yonghui Qin, Boqian Lin, Dongyue Chen, and Tong Jia, *Member, IEEE*

***Abstract*—Few-shot time series classification (TSC) remains challenging because, under extremely limited supervision, a model must not only extract stable class-discriminative patterns from a handful of labeled examples, but also develop a decision mechanism that generalizes to unseen classes. Existing methods typically model time series from a single view, making it difficult to jointly capture local-to-global temporal dependencies and holistic waveform morphology. Meanwhile, although pretrained large language models (LLMs) offer strong conditional generative priors, their effective use in TSC is hindered by the representation gap between continuous-valued time series and the hidden space of language models. Moreover, if prediction is still performed with a conventional discriminative classification head, the generative decision-making capability of LLMs—aligned with their pretraining objective—cannot be fully exploited. To address these issues, we propose a unified generative framework for few-shot TSC. Our method first employs a dual-view time-series encoder, where a temporal branch and a visual branch extract complementary dynamic dependencies and morphological structure patterns, respectively. The resulting representations are then aligned to the hidden space of a frozen LLM through alignment pretraining. Finally, we reformulate classification as constrained generation of class-specific tokens, enabling the model to leverage the conditional generation capability of the LLM for few-shot decision-making, while constrained decoding further reduces prediction uncertainty in low-data regimes. Experiments under a strict few-shot protocol on UCR benchmarks show that the proposed method consistently outperforms existing deep learning baselines across multiple shot settings. Ablation studies further confirm the complementarity of the dual-view design, the stable gains and faster convergence brought by alignment pretraining, and the critical role of constrained decoding in generative few-shot classification.**

***Index Terms*—Time-Series Classification, Few-Shot Learning, Generative Classification, Large Language Models, Prompting**

I. Introduction

Time series classification (TSC) is a core problem in machine learning, underpinning applications such as healthcare monitoring, industrial diagnosis, human activity recognition, and scientific sensing [1]. While deep neural networks have substantially advanced fully supervised TSC, their success typically depends on having sufficiently many labeled sequences [2]. In many practical settings, however, annotations are scarce, costly, or slow to acquire—for example in rare fault diagnosis, patient-specific monitoring, and newly deployed sensing systems. Few-shot TSC therefore remains a practically important and fundamentally difficult regime: the model must infer class-discriminative structure from only a handful of labeled examples per class while remaining robust to large intra-class variation and cross-domain heterogeneity [3].

Existing few-shot TSC methods have mainly explored meta-learning, prototype- or shapelet-based reasoning, and spectral or contrastive representation learning [3]–[5]. In parallel, recent work has begun to connect pretrained large language models (LLMs) to time-series analysis through quantization, projection, prompt-based adaptation, and multimodal interfaces [6]–[8]. Yet these two lines have not fully met where few-shot TSC is hardest. Current LLM-based pipelines often rely on a single representation pathway, even though recent work increasingly suggests that visualized time-series signals can expose morphology-aware cues that are complementary to sequence-space modeling [9], [10]. They also face a persistent modality gap between continuous temporal features and the hidden space of frozen language models [8], [11]. Moreover, final decisions are frequently made either by external discriminative heads or by unconstrained open-vocabulary generation, both of which are only weakly aligned with the autoregressive objective that causal LLMs are pretrained to solve.

We take the view that effective few-shot TSC with frozen causal LLMs is bottlenecked by three coupled issues: representation, alignment, and decision. First, under extreme label scarcity, a single-view encoder may fail to recover all class-discriminative cues from the input sequence. Second, even strong temporal features are of limited use unless they can be mapped into a latent interface that the frozen LLM can reliably exploit. Third, if the prediction mechanism is mismatched with autoregressive next-token generation, the model may underuse the time-series information already encoded in its internal states. Recent evidence that prompt outputs can substantially underestimate the time-series classification capability contained in LLM representations further reinforces this point [12].

Motivated by this perspective, we propose a dual-view generative framework for few-shot TSC. A patch-based temporal encoder, inspired by patchified time-series Transformers [13], captures local-to-global temporal dependencies directly in sequence space. In parallel, a visual branch renders each time series as a 2D image and extracts morphology-aware representations that emphasize waveform shape, texture, periodic

This work was supported in part by the National Key R&D Program of China under Grant 2024YFB4710900, in part by the Guangdong Basic and Applied Basic Research Foundation under Grant 2024A1515010244, in part by the National Natural Science Foundation of China under Grant U25A20477, and in part by the 111 Project under Grant B16009. (Corresponding authors: Dongyue Chen and Xinde Li.)

Shizhuo Deng and Dongyue Chen are with the College of Information Science and Engineering, Key Laboratory of Data Analytics and Optimization for Smart Industry, Northeastern University, Shenyang 110819, China, and also with the Foshan Graduate School of Innovation, Northeastern University, Foshan 528311, China (e-mail: dengshizhuo@mail.neu.edu.cn; chendongyue@ise.neu.edu.cn).

Yonghui Qin, Boqian Lin, and Tong Jia are with the College of Information Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: 2400885@stu.neu.edu.cn; lin.boqian@foxmail.com; jiatong@ise.neu.edu.cn).

structure, and spatialized anomalies. Rather than collapsing the two views prematurely, we preserve their token-level outputs and concatenate them before projection, so that the downstream model can condition jointly on temporal dynamics and morphological structure.

To make this dual-view evidence usable by a frozen causal LLM, we project the concatenated temporal tokens into the LLM hidden space and treat them as soft prompt tokens inserted into an instruction-style input. We further adopt a stage-wise alignment strategy before few-shot supervision. The first stage learns to produce LLM-compatible time-series representations and reduces the modality gap between temporal features and language-model hidden states. The second stage performs joint optimization of the dual-view encoder, the projection module, and lightweight LoRA adapters while keeping the main LLM backbone frozen. This design makes alignment a primary training target rather than an incidental by-product of few-shot finetuning, which is especially important when only a small support set is available.

Finally, instead of attaching an external classifier, we reformulate few-shot TSC as constrained label-token generation. We introduce class-specific tokens into the tokenizer and train the model to generate the correct label token conditioned on textual instructions and dual-view time-series soft prompts. During inference, decoding is restricted to the candidate class tokens in the current episode together with the end-of-sequence token. This preserves consistency with the causal LLM's native generation objective while removing irrelevant vocabulary degrees of freedom, thereby yielding more stable decisions under few-shot uncertainty.

Extensive experiments under a strict support/query protocol validate this formulation. Across shot settings, the dual-view model consistently outperforms either single branch alone; stage-wise alignment pretraining accelerates convergence, stabilizes optimization, and improves final accuracy; constrained decoding materially boosts prediction quality; and the overall framework surpasses strong deep-learning baselines at the same shot level. These results suggest that the key to few-shot TSC is not simply introducing a larger pretrained model, but enabling the model to see richer temporal evidence, align that evidence with a frozen LLM in latent space, and decide through a generation mechanism tailored to the episode label set.

In summary, our contributions are threefold: (1) we introduce a dual-view encoder that couples patch-based temporal modeling with morphology-aware visual modeling for few-shot TSC; (2) we present a stage-wise time-series-to-LLM alignment scheme with parameter-efficient adaptation for frozen causal LLMs; and (3) we cast few-shot TSC as constrained label-token generation and show that dual-view encoding, alignment pretraining, and constrained decoding each make measurable contributions to performance.

II. Related Work

a) Few-shot time series classification.: Few-shot time series classification (TSC) has mainly been studied through meta-learning, metric learning, and prototype-based inductive biases. Narwariya et al. [14] introduced a meta-learning framework that trains residual networks across heterogeneous UCR tasks for rapid adaptation under scarce supervision. Tang et al. [3] proposed Dual Prototypical Shapelet Networks (DPSN), which combine representative class prototypes with discriminative local shapelets to improve both interpretability and data efficiency. Yang et al. [5] further explored spectral relations and introduced the Spectral Propagation Graph Network (SPGN) to propagate class information through spectrum-wise comparisons. More recently, COSCO [4] improved few-shot multivariate TSC from the optimization perspective by combining sharpness-aware minimization with a prototypical loss. These methods demonstrate the importance of transferable priors and structured inductive biases, but they remain essentially discriminative pipelines and mostly operate within a single temporal representation space.

b) LLMs and foundation models for time series.: Recent surveys show that foundation-model research for time series has rapidly expanded from adapting pretrained language models to building native temporal foundation models [15]–[17]. Representative adaptation-based approaches include TEST [18], which aligns time-series embeddings with the LLM embedding space through text-prototype-aware contrastive learning; Time-LLM [7], which reprograms time-series patches into a frozen LLM via text prototypes and prompt-as-prefix; and S^2IP-LLM [19], which aligns time-series embeddings with semantic anchors derived from pretrained word embeddings. LLM4TS [11] further adopts a two-stage alignment-and-task fine-tuning strategy, while OpenTSLM [20] moves toward native time-series language models with soft-prompt and cross-attention interfaces. A recent line of work has also begun to study few-shot classification with LLM-based architectures. The closest to our setting is LLMFew [21], which uses a patch-wise temporal encoder, LoRA-tuned decoder, and a conventional classification head for few-shot multivariate TSC. Compared with this line, we focus on dual-view temporal evidence, explicit stage-wise alignment before few-shot adaptation, and generative label-token prediction rather than an external classifier. We are also motivated by recent ablation studies suggesting that naively attaching time-series inputs to pretrained LLMs is insufficient unless the temporal interface is meaningfully designed and aligned [22].

c) Vision-based representations for time series.: Beyond sequence-space modeling, a growing body of work represents time series as images so that waveform morphology, texture, periodicity, and holistic structural patterns can be captured by vision backbones. Recent surveys identify this as an increasingly important branch of time-series foundation modeling [9]. In the context of multimodal reasoning, Liu et al. [10] show that visualization can substantially improve LLM reasoning over time series. For classification, TiViT [23] demonstrates that pretrained vision transformers applied to imaged time series can yield highly competitive representations and are complementary to time-series foundation models. These results suggest that visualized signals expose discriminative cues that are not fully captured by sequence-only modeling. However, existing visual approaches are typically used as alternatives to temporal encoders or as interfaces for reasoning,

rather than being fused at the token level with a temporal branch for few-shot generative classification.

d) Generative classification and constrained decoding.: Casting classification as generation has been widely explored in NLP, most notably in the text-to-text paradigm of T5 [24]. Rather than relying on an external discriminative head, this paradigm formulates prediction as conditional token generation, making the decision process more naturally aligned with the pretrained objective of language models. In parallel, constrained decoding has been studied as a general mechanism for restricting generation to valid outputs under lexical or structural constraints [25], [26]. Inspired by these ideas, we formulate few-shot time series classification as generation of class-specific label tokens conditioned on textual instructions and time-series soft prompts. Instead of attaching a separate classifier, our model performs prediction in the native autoregressive manner of a frozen causal LLM, while restricting decoding to the candidate label space of the current episode further reduces irrelevant output freedom and improves decision stability under low-data uncertainty.

In summary, our method lies at the intersection of these four lines of work: few-shot TSC, LLM-based time-series adaptation, vision-based time-series representation learning, and generative classification with constrained decoding. It differs from prior work by jointly combining dual-view token-level encoding, stage-wise time-series-to-LLM alignment, and episode-wise constrained label-token generation in a unified framework for few-shot TSC.

III. Method

A. Problem Setup

We study few-shot time series classification (TSC) under an episodic setting. Let $\mathcal{C}$ denote the label space of a dataset. In each episode e, we sample a class subset $\mathcal{C}^{(e)} \subseteq \mathcal{C}$ and construct a support set $\mathcal{S}^{(e)} = \{(x_i, y_i)\}_{i=1}^{N_s}$ and a query set $\mathcal{Q}^{(e)} = \{(x_j, y_j)\}_{j=1}^{N_q}$ such that $y_i, y_j \in \mathcal{C}^{(e)}$. Each input $x \in \mathbb{R}^L$ is a univariate time series of length L. The goal is to predict the class label of each query example using only the limited labeled support examples in the current episode.

Instead of learning a conventional discriminative classifier, we reformulate few-shot TSC as *constrained label-token generation*. For each class $c \in \mathcal{C}$, we introduce a class-specific token g_c into the tokenizer vocabulary. Given an input time series and a textual instruction, the model is trained to generate the correct label token g_y followed by an end-of-sequence token. During inference, generation is restricted to the candidate label tokens in the current episode. This turns few-shot TSC into conditional generation over an episode-specific label space.

B. Framework Overview

Our framework consists of four components: (1) a dual-view time-series encoder, (2) a lightweight projector that maps time-series features into the hidden space of a frozen causal LLM, (3) a parameter-efficient adaptation module based on LoRA, and (4) a generative decision layer implemented through class-specific token prediction.

Given a time series x, we encode it from two complementary views. A temporal branch captures local-to-global temporal dependencies directly in sequence space, while a visual branch converts the series into a 2D image and extracts morphology-aware representations. Unlike architectures that aggressively compress multimodal evidence into a small number of slots, we preserve the token-level outputs of both branches and directly concatenate them before projection. The projected tokens are then injected into an instruction-style input as continuous soft prompts for a frozen causal LLM, which predicts a class token autoregressively.

C. Dual-View Time-Series Encoder

a) Temporal branch.: We first partition the input series x into overlapping patches of length P with stride S, yielding M_t temporal patches:

$$\mathbf{p}_m = x_{(m-1)S+1:(m-1)S+P}, \qquad m = 1, \ldots, M_t.$$

Each patch is linearly embedded and augmented with positional information, and the resulting patch sequence is fed into a Transformer encoder:

$$\mathbf{H}^t = F_t(\mathbf{p}_1, \ldots, \mathbf{p}_{M_t}) \in \mathbb{R}^{M_t \times d_{ts}},$$

where F_t denotes the temporal encoder and d_{ts} is the time-series feature dimension. This branch is designed to model short-range local patterns, long-range dependencies, and temporal evolution dynamics in the original 1D signal space.

b) Visual branch.: In parallel, we transform the same time series into a 2D visual representation:

$$\mathbf{I} = \mathcal{R}(x),$$

where $\mathcal{R}(\cdot)$ is a deterministic rendering function that converts the 1D signal into an image. The rendered image is processed by a visual backbone F_v, producing a sequence of visual tokens:

$$\widetilde{\mathbf{H}}^v = F_v(\mathbf{I}) \in \mathbb{R}^{M_v \times d_v}.$$

Because the visual backbone and the temporal branch may operate in different feature spaces, we project the visual tokens into the same time-series feature dimension:

$$\mathbf{H}^v = \widetilde{\mathbf{H}}^v \mathbf{W}_v + \mathbf{b}_v \in \mathbb{R}^{M_v \times d_{ts}}.$$

This branch emphasizes morphology-aware cues such as waveform shape, periodic structure, texture-like patterns, and spatialized anomalies that may be less explicit in a purely sequential representation.

c) Token-level fusion.: The two views are fused by direct concatenation at the token level:

$$\mathbf{H} = [\mathbf{H}^t; \mathbf{H}^v] \in \mathbb{R}^{(M_t + M_v) \times d_{ts}}.$$

Importantly, we do not apply slot-based pooling or early aggregation before fusion. This preserves fine-grained evidence from both views and allows the downstream LLM to attend jointly to temporal dynamics and morphology-aware structure.

D. Time-Series-to-LLM Interface

The dual-view encoder outputs are not converted into textualized numbers. Instead, they are mapped into the hidden space of a causal LLM through a lightweight projector:

$$\mathbf{Z} = \mathbf{H}\mathbf{W}_p + \mathbf{b}_p \in \mathbb{R}^{M \times d_{llm}},$$

where $M = M_t + M_v$ and d_{llm} is the hidden size of the LLM.

We then build an instruction-style input for each sample. The textual part contains: (1) a task instruction stating that the problem is an N-class classification task and that the model should output only a class token, (2) a short description specifying the time-series modality and length information, and (3) a suffix prompt that asks for the answer. Let $\mathbf{E}_{pre}$, $\mathbf{E}_{desc}$, and $\mathbf{E}_{suf}$ denote the corresponding text embeddings produced by the tokenizer and embedding layer of the LLM. The final conditioning sequence is assembled as

$$\mathbf{C}(x) = [\mathbf{E}_{pre}; \mathbf{E}_{desc}; \mathbf{Z}; \mathbf{E}_{suf}].$$

Hence, the LLM receives a mixed sequence composed of text tokens and continuous time-series soft prompts.

E. Generative Classification with Class Tokens

For each class c, we add a unique label token g_c to the tokenizer. Given an example (x, y), the target answer sequence is

$$\mathbf{a}(y) = [g_y, \texttt{<eos>}].$$

During training, the answer embeddings are appended to the conditioning sequence:

$$\mathbf{X}(x, y) = [\mathbf{C}(x); \mathbf{E}(g_y); \mathbf{E}(\texttt{<eos>})].$$

The model is optimized with an autoregressive language-modeling objective, but the loss is computed only on the answer segment:

$$\mathcal{L}_{gen} = -\log p_\Theta(g_y \mid \mathbf{C}(x)) - \log p_\Theta(\texttt{<eos>} \mid \mathbf{C}(x), g_y).$$

Therefore, the textual prompt and the time-series soft prompts serve purely as conditions, while supervision is concentrated on the generated label token and the end token.

In practice, we allow the token embedding matrix and the LM head to be trainable for implementation simplicity. However, because supervision is applied only to the answer segment, the optimization pressure is primarily focused on the class-specific tokens and their surrounding decision interface.

F. Constrained Decoding

At inference time, the target answer is not appended. Given the conditioning sequence $\mathbf{C}(x)$, the model generates at most two tokens. To reduce irrelevant output freedom, decoding is constrained to the episode-specific candidate set

$$\mathcal{V}^{(e)} = \{g_c \mid c \in \mathcal{C}^{(e)}\} \cup \{\texttt{<eos>}\}.$$

Because each class is represented by a single token, prediction is equivalent to

$$\hat{y} = \arg\max_{c \in \mathcal{C}^{(e)}} p_\Theta(g_c \mid \mathbf{C}(x)).$$

The model then emits <eos> after the selected class token. This constrained decoding strategy preserves consistency with the autoregressive objective while removing irrelevant open-vocabulary alternatives, leading to more stable decisions in low-data regimes.

G. Stage-wise Alignment and Parameter-Efficient Adaptation

A central challenge in our framework is the representation mismatch between time-series features and the latent space of the frozen LLM. To address this issue, we adopt a stage-wise optimization strategy.

a) Stage 1: alignment warm-up.: In the first stage, we optimize the dual-view encoder and the projector while keeping the main LLM backbone frozen and without activating LoRA. This stage focuses on learning an LLM-compatible interface for the injected time-series tokens and stabilizes cross-modal alignment before joint adaptation.

b) Stage 2: joint adaptation.: In the second stage, we continue training from the warm-up initialization and activate LoRA modules in the LLM. The trainable parameters then include the dual-view encoder, the projector, the LoRA parameters, and the class-token-related embedding/output parameters, while the original LLM backbone remains frozen. This design preserves the pretrained generative prior of the LLM while allowing parameter-efficient task adaptation.

When alignment pretraining is used, the same stage-wise schedule is first applied on the pretraining data to initialize the time-series-to-LLM interface, and the resulting weights are then adapted to downstream few-shot episodes. When alignment pretraining is disabled, the model is trained directly on the downstream task using the same generative objective.

H. Few-Shot Episodic Adaptation

For few-shot learning, training is conducted only on the support set of each episode. We consider both all-way K-shot episodes and N-way K-shot episodes. For the latter, we first sample N classes and then sample up to K support examples per class. If a class contains fewer than K training examples, we use all available examples of that class rather than dropping it. The query set is formed from test samples belonging to the same episode-specific class subset $\mathcal{C}^{(e)}$.

This protocol makes each run a self-contained episode with its own support set, query set, and decoding label space. The final model is evaluated directly on the query set of the same episode, ensuring that both supervision and constrained decoding are matched to the current few-shot task.

I. Full-Supervised Variant

The same architecture also supports a full-supervised training regime. This variant can be obtained by removing episodic sampling, training on the full training split, and performing constrained decoding over the complete dataset label space. Therefore, the difference between the few-shot and full-supervised settings lies in the training protocol rather than in the model architecture itself.

IV. Other Resources

See [**?**], [**?**], [**?**], [**?**], [**?**] for resources on formatting math into text and additional help in working with LaTeX.

V. Text

For some of the remainer of this sample we will use dummy text to fill out paragraphs rather than use live text that may violate a copyright.

Itam, que ipiti sum dem velit la sum et dionet quatibus apitet voloritet audam, qui aliciant voloreicid quaspe volorem ut maximusandit faccum conemporerum aut ellatur, nobis arcimus. Fugit odi ut pliquia incitium latum que cusapere perit molupta eaquaeria quod ut optatem poreiur? Quiaerr ovitior suntiant litio bearciur?

Onseque sequaes rectur autate minullore nusae nestiberum, sum voluptatio. Et ratem sequiam quaspername nos rem repudandae volum consequis nos eium aut as molupta tectum ulparumquam ut maximillesti consequas quas inctia cum volectinusa porrum unt eius cusaest exeritatur? Nias es enist fugit pa vollum reium essusam nist et pa aceaqui quo elibusdandis deligendus que nullaci lloreri bla que sa coreriam explacc atiumquos simolorpore, non prehendunt lam que occum [**?**] si aut aut maximus eliaeruntia dia sequiamenime natem sendae ipidemp orehend uciisi omnienetus most verum, ommolendi omnimus, est, veni aut ipsa volendelist mo conserum volores estisciis recessi nveles ut poressitatur sitiis ex endi diti volum dolupta aut aut odi as eatquo cullabo remquis toreptum et des accus dolende pores sequas dolores tinust quas expel moditae ne sum quiatis nis endipie nihilis etum fugiae audi dia quiasit quibus. Ibus el et quatemo luptatque doluptaest et pe volent rem ipidusa eribus utem venimolorae dera qui acea quam etur aceruptat. Gias anis doluptaspic tem et aliquis alique inctiuntiur?

Sedigent, si aligend elibuscid ut et ium volo tem eictore pellore ritatus ut ut ullatus in con con pere nos ab ium di tem aliqui od magnit repta volectur suntio. Nam isquiante doluptis essit, ut eos suntionsecto debitiur sum ea ipitiis adipit, oditiore, a dolorerempos aut harum ius, atquat.

Rum rem ditinti sciendunti volupiciendi sequiae nonsect oreniatur, volores sition ressimil inus solut ea volum harumqui to see(1) mint aut quat eos explis ad quodi debis deliqui aspel earcius.

$$x = \sum_{i=0}^{n} 2iQ. \tag{1}$$

Alis nime volorempera perferi sitio denim repudae pre ducilit atatet volecte ssimillorae dolore, ut pel ipsa nonsequiam in re nus maiost et que dolor sunt eturita tibusanis eatent a aut et dio blaudit reptibu scipitem liquia consequodi od unto ipsae. Et enitia vel et experferum quiat harum sa net faccae dolut voloria nem. Bus ut labo. Ita eum repraer rovitia samendit aut et volupta tecupti busant omni quiae porro que nossimodic temquis anto blacita conse nis am, que ereperum eumquam quaescil imenisci quae magnimos recus ilibeaque cum etum iliate prae parumquatemo blaceaquiam quundia dit apienditem rerit re eici quaes eos sinvers pelecabo. Namendignis as exerupit aut magnim ium illabor roratecte plic tem res apiscipsam et vernat untur a deliquaest que non cus eat ea dolupiducim fugiam volum hil ius dolo eaquis sitis aut landesto quo corerest et auditaquas ditae voloribus, qui optaspis exero cusa am, ut plibus.

VI. Some Common Elements

A. Sections and Subsections

Enumeration of section headings is desirable, but not required. When numbered, please be consistent throughout the article, that is, all headings and all levels of section headings in the article should be enumerated. Primary headings are designated with Roman numerals, secondary with capital letters, tertiary with Arabic numbers; and quaternary with lowercase letters. Reference and Acknowledgment headings are unlike all other section headings in text. They are never enumerated. They are simply primary headings without labels, regardless of whether the other headings in the article are enumerated.

B. Citations to the Bibliography

The coding for the citations is made with the LaTeX `\cite` command. This will display as: see [**?**].

For multiple citations code as follows: `\cite{ref1,ref2,ref3}` which will produce [**?**], [**?**], [**?**]. For reference ranges that are not consecutive code as `\cite{ref1,ref2,ref3,ref9}` which will produce [**?**], [**?**], [**?**], [**?**]

C. Lists

In this section, we will consider three types of lists: simple unnumbered, numbered, and bulleted. There have been many options added to IEEEtran to enhance the creation of lists. If your lists are more complex than those shown below, please refer to the original "IEEEtran_HOWTO.pdf" for additional options.

A plain unnumbered list:

bare_jrnl.tex
bare_conf.tex
bare_jrnl_compsoc.tex
bare_conf_compsoc.tex
bare_jrnl_comsoc.tex

A simple numbered list:

1) bare_jrnl.tex
2) bare_conf.tex
3) bare_jrnl_compsoc.tex
4) bare_conf_compsoc.tex
5) bare_jrnl_comsoc.tex

A simple bulleted list:

- bare_jrnl.tex
- bare_conf.tex
- bare_jrnl_compsoc.tex
- bare_conf_compsoc.tex
- bare_jrnl_comsoc.tex

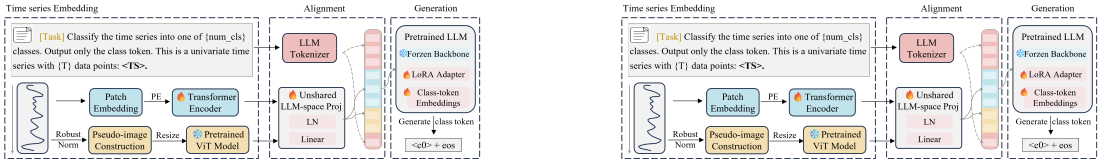

Fig. 1. Dae. Ad quatur autat ut porepel itemoles dolor autem fuga. Bus quia con nessunti as remo di quatus non perum que nimus. (a) Case I. (b) Case II.

TABLE I
AN EXAMPLE OF A TABLE

One	Two
Three	Four

D. Figures

Fig. 1 is an example of a floating figure using the graphicx package. Note that `\label` must occur AFTER (or within) `\caption`. For figures, `\caption` should occur after the `\includegraphics`.

Fig. 2(a) and 2(b) is an example of a double column floating figure using two subfigures. (The subfig.sty package must be loaded for this to work.) The subfigure `\label` commands are set within each subfloat command, and the `\label` for the overall figure must come after `\caption`. `\hfil` is used as a separator to get equal spacing. The combined width of all the parts of the figure should do not exceed the text width or a line break will occur.

Note that often IEEE papers with multi-part figures do not place the labels within the image itself (using the optional argument to `\subfloat[]`), but instead will reference/describe all of them (a), (b), etc., within the main caption. Be aware that for subfig.sty to generate the (a), (b), etc., subfigure labels, the optional argument to `\subfloat` must be present. If a subcaption is not desired, leave its contents blank, e.g.,`\subfloat[]`.

VII. TABLES

Note that, for IEEE-style tables, the `\caption` command should come BEFORE the table. Table captions use title case. Articles (a, an, the), coordinating conjunctions (and, but, for, or, nor), and most short prepositions are lowercase unless they are the first or last word. Table text will default to `\footnotesize` as the IEEE normally uses this smaller font for tables. The `\label` must come after `\caption` as always.

VIII. ALGORITHMS

Algorithms should be numbered and include a short title. They are set off from the text with rules above and below the title and after the last line.

Algorithm 1 Weighted Tanimoto ELM.

TRAIN($\mathbf{XT}$)
  select randomly $W \subset \mathbf{X}$
  $N_{\mathbf{t}} \leftarrow |\{i : \mathbf{t}_i = \mathbf{t}\}|$ **for** $\mathbf{t} = -1, +1$
  $B_i \leftarrow \sqrt{\text{MAX}(N_{-1}, N_{+1})/N_{\mathbf{t}_i}}$ **for** $i = 1, ..., N$
  $\hat{\mathbf{H}} \leftarrow B \cdot (\mathbf{X}^T\mathbf{W})/(\not\Vdash\mathbf{X} + \not\Vdash\mathbf{W} - \mathbf{X}^T\mathbf{W})$
  $\beta \leftarrow \left(I/C + \hat{\mathbf{H}}^T\hat{\mathbf{H}}\right)^{-1} (\hat{\mathbf{H}}^T B \cdot \mathbf{T})$
  return $\mathbf{W}, \beta$

PREDICT($\mathbf{X}$)
  $\mathbf{H} \leftarrow (\mathbf{X}^T\mathbf{W})/(\not\Vdash\mathbf{X} + \not\Vdash\mathbf{W} - \mathbf{X}^T\mathbf{W})$
  return SIGN($\mathbf{H}\beta$)

Que sunt eum lam eos si dic to estist, culluptium quid qui nestrum nobis reiumquiatur minimus minctem. Ro moluptat fuga. Itatquiam ut laborpo rersped exceres vollandi repudaerem. Ulparci sunt, qui doluptaquis sumquia ndestiu sapient iorepella sunti veribus. Ro moluptat fuga. Itatquiam ut laborpo rersped exceres vollandi repudaerem.

IX. MATHEMATICAL TYPOGRAPHY AND WHY IT MATTERS

Typographical conventions for mathematical formulas have been developed to **provide uniformity and clarity of presentation across mathematical texts**. This enables the readers of those texts to both understand the author's ideas and to grasp new concepts quickly. While software such as LaTeX and MathType® can produce aesthetically pleasing math when used properly, it is also very easy to misuse the software, potentially resulting in incorrect math display.

IEEE aims to provide authors with the proper guidance on mathematical typesetting style and assist them in writing the best possible article. As such, IEEE has assembled a set of examples of good and bad mathematical typesetting [**?**], [**?**], [**?**], [**?**], [**?**].

Further examples can be found at http://journals.ieeeauthorcenter.ieee.org/wp-content/uploads/sites/7/IEEE-Math-Typesetting-Guide-for-LaTeX-Users.pdf

A. Display Equations

The simple display equation example shown below uses the "equation" environment. To number the equations, use the `\label` macro to create an identifier for the equation. LaTeX will automatically number the equation for you.

$$x = \sum_{i=0}^{n} 2iQ. \tag{2}$$

is coded as follows:

```
\begin{equation}
\label{deqn_ex1}
x = \sum_{i=0}^{n} 2{i} Q.
\end{equation}
```

To reference this equation in the text use the `\ref` macro. Please see (2)
is coded as follows:

```
Please see (\ref{deqn_ex1})
```

B. Equation Numbering

Consecutive Numbering: Equations within an article are numbered consecutively from the beginning of the article to the end, i.e., (1), (2), (3), (4), (5), etc. Do not use roman numerals or section numbers for equation numbering.
Appendix Equations: The continuation of consecutively numbered equations is best in the Appendix, but numbering as (A1), (A2), etc., is permissible.

Hyphens and Periods: Hyphens and periods should not be used in equation numbers, i.e., use (1a) rather than (1-a) and (2a) rather than (2.a) for subequations. This should be consistent throughout the article.

C. Multi-Line Equations and Alignment

Here we show several examples of multi-line equations and proper alignments.
A single equation that must break over multiple lines due to length with no specific alignment.

$$\begin{multline}\text{The first line of this example}\\ \text{The second line of this example}\\ \text{The third line of this example}\end{multline} \tag{3}$$

is coded as:

```
\begin{multline}
\text{The first line of this example}\\
\text{The second line of this example}\\
\text{The third line of this example}
\end{multline}
```

A single equation with multiple lines aligned at the = signs

$$a = c + d \tag{4}$$
$$b = e + f \tag{5}$$

is coded as:

```
\begin{align}
a &= c+d \\
b &= e+f
\end{align}
```

The `align` environment can align on multiple points as shown in the following example:

$$x = y \qquad X = Y \qquad a = bc \tag{6}$$
$$x' = y' \qquad X' = Y' \qquad a' = bz \tag{7}$$

is coded as:

```
\begin{align}
x &= y & X & =Y & a &=bc\\
x' &= y' & X' &=Y' &a' &=bz
\end{align}
```

D. Subnumbering

The amsmath package provides a `subequations` environment to facilitate subnumbering. An example:

$$f = g \tag{8a}$$
$$f' = g' \tag{8b}$$
$$\mathcal{L}f = \mathcal{L}g \tag{8c}$$

is coded as:

```
\begin{subequations}\label{eq:2}
\begin{align}
f&=g \label{eq:2A}\\
f' &=g' \label{eq:2B}\\
\mathcal{L}f &= \mathcal{L}g \label{eq:2c}
\end{align}
\end{subequations}
```

E. Matrices

There are several useful matrix environments that can save you some keystrokes. See the example coding below and the output.
A simple matrix:

$$\begin{matrix} 0 & 1 \\ 1 & 0 \end{matrix} \tag{9}$$

is coded as:

```
\begin{equation}
\begin{matrix}  0 &  1 \\
1 &  0 \end{matrix}
\end{equation}
```

A matrix with parenthesis

$$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \tag{10}$$

is coded as:

```
\begin{equation}
\begin{pmatrix} 0 & -i \\
 i &  0 \end{pmatrix}
\end{equation}
```

A matrix with square brackets

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \tag{11}$$

is coded as:

```
\begin{equation}
\begin{bmatrix} 0 & -1 \\
1 &  0 \end{bmatrix}
```

```
\end{equation}
```

A matrix with curly braces

$$\begin{Bmatrix} 1 & 0 \\ 0 & -1 \end{Bmatrix} \tag{12}$$

is coded as:

```
\begin{equation}
\begin{Bmatrix} 1 &  0 \\
0 & -1 \end{Bmatrix}
\end{equation}
```

A matrix with single verticals

$$\begin{vmatrix} a & b \\ c & d \end{vmatrix} \tag{13}$$

is coded as:

```
\begin{equation}
\begin{vmatrix} a &  b \\
c &  d \end{vmatrix}
\end{equation}
```

A matrix with double verticals

$$\begin{Vmatrix} i & 0 \\ 0 & -i \end{Vmatrix} \tag{14}$$

is coded as:

```
\begin{equation}
\begin{Vmatrix} i &  0 \\
0 & -i \end{Vmatrix}
\end{equation}
```

F. Arrays

The `array` environment allows you some options for matrix-like equations. You will have to manually key the fences, but there are other options for alignment of the columns and for setting horizontal and vertical rules. The argument to `array` controls alignment and placement of vertical rules.

A simple array

$$\left(\begin{array}{cccc} a+b+c & uv & x-y & 27 \\ a+b & u+v & z & 134 \end{array} \right) \tag{15}$$

is coded as:

```
\begin{equation}
\left(
\begin{array}{cccc}
a+b+c & uv & x-y & 27\\
a+b & u+v & z & 134
\end{array} \right)
\end{equation}
```

A slight variation on this to better align the numbers in the last column

$$\left(\begin{array}{cccc} a+b+c & uv & x-y & 27 \\ a+b & u+v & z & 134 \end{array} \right) \tag{16}$$

is coded as:

```
\begin{equation}
\left(
\begin{array}{cccr}
a+b+c & uv & x-y & 27\\
a+b & u+v & z & 134
\end{array} \right)
\end{equation}
```

An array with vertical and horizontal rules

$$\left(\begin{array}{c|c|c|c} a+b+c & uv & x-y & 27 \\ \hline a+b & u+v & z & 134 \end{array} \right) \tag{17}$$

is coded as:

```
\begin{equation}
\left(
\begin{array}{c|c|c|r}
a+b+c & uv & x-y & 27\\
a+b & u+v & z & 134
\end{array} \right)
\end{equation}
```

Note the argument now has the pipe "|" included to indicate the placement of the vertical rules.

G. Cases Structures

Many times cases can be miscoded using the wrong environment, i.e., `array`. Using the `cases` environment will save keystrokes (from not having to type the `\left\lbrace`) and automatically provide the correct column alignment.

$$z_m(t) = \begin{cases} 1, & \text{if } \beta_m(t) \\ 0, & \text{otherwise.} \end{cases}$$

is coded as follows:

```
\begin{equation*}
{z_m(t)} =
\begin{cases}
1,&{\text{if}}\ {\beta }_m(t),\\
{0,}&{\text{otherwise.}}
\end{cases}
\end{equation*}
```

Note that the "&" is used to mark the tabular alignment. This is important to get proper column alignment. Do not use `\quad` or other fixed spaces to try and align the columns. Also, note the use of the `\text` macro for text elements such as "if" and "otherwise."

H. Function Formatting in Equations

Often, there is an easy way to properly format most common functions. Use of the `\` in front of the function name will in most cases, provide the correct formatting. When this does not work, the following example provides a solution using the `\text` macro:

$$d_R^{KM} = \arg\min_{d_l^{KM}} \{d_1^{KM}, \ldots, d_6^{KM}\}.$$

is coded as follows:

```
\begin{equation*}
 d_{R}^{KM} = \underset {d_{l}^{KM}}
 {\text{arg min}} \{ d_{1}^{KM},
 \ldots,d_{6}^{KM}\}.
\end{equation*}
```

I. Text Acronyms Inside Equations

This example shows where the acronym "MSE" is coded using \text{} to match how it appears in the text.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

```
\begin{equation*}
 \text{MSE} = \frac {1}{n}\sum _{i=1}^{n}
(Y_{i} - \hat {Y_{i}})^{2}
\end{equation*}
```

X. Conclusion

The conclusion goes here.

Acknowledgments

This should be a simple paragraph before the References to thank those individuals and institutions who have supported your work on this article.

Appendix
Proof of the Zonklar Equations

Use \appendix if you have a single appendix: Do not use \section anymore after \appendix, only \section*. If you have multiple appendixes use \appendices then use \section to start each appendix. You must declare a \section before using any \subsection or using \label (\appendices by itself starts a section numbered zero.)

References

[1] H. Ismail Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: a review," *Data mining and knowledge discovery*, vol. 33, no. 4, pp. 917–963, 2019.

[2] S. Pan, H. Luo, M. C. Chua, K. Pugalenthi, and E. K. Yapp, "A review of similarity-based few-shot learning methods for time series classification in manufacturing," in *2024 6th International Conference on Industrial Artificial Intelligence (IAI)*. IEEE, 2024, pp. 1–6.

[3] W. Tang, L. Liu, and G. Long, "Interpretable time-series classification on few-shot samples," in *2020 International joint conference on neural networks (IJCNN)*. IEEE, 2020, pp. 1–8.

[4] J. Barreda, A. Gomez, R. Puga, K. Zhou, and L. Zhang, "Cosco: a sharpness-aware training framework for few-shot multivariate time series classification," in *Proceedings of the 33rd ACM international conference on information and knowledge management*, 2024, pp. 3622–3626.

[5] L. Yang, S. Hong, and L. Zhang, "Spectral propagation graph network for few-shot time series classification," *arXiv preprint arXiv:2202.04769*, 2022.

[6] X. Zhang, R. R. Chowdhury, R. K. Gupta, and J. Shang, "Large language models for time series: A survey," *arXiv preprint arXiv:2402.01801*, 2024.

[7] M. Jin, S. Wang, L. Ma, Z. Chu, J. Zhang, X. Shi, P.-Y. Chen, Y. Liang, Y.-f. Li, S. Pan *et al.*, "Time-llm: Time series forecasting by reprogramming large language models," in *International Conference on Learning Representations*, 2024.

[8] M. Cheng, Y. Chen, Q. Liu, Z. Liu, and Y. Luo, "Advancing time series classification with multimodal language modeling," *arXiv preprint arXiv:2403.12371*, 2024.

[9] J. Ni, Z. Zhao, C. Shen, H. Tong, D. Song, W. Cheng, D. Luo, and H. Chen, "Harnessing vision models for time series analysis: a survey," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, 2025, pp. 10 612–10 620.

[10] H. Liu, C. Liu, and B. A. Prakash, "A picture is worth a thousand numbers: Enabling llms reason about time series via visualization," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 7486–7518.

[11] C. Chang, W.-Y. Wang, W.-C. Peng, and T.-F. Chen, "Llm4ts: Aligning pre-trained llms as data-efficient time-series forecasters," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 3, pp. 1–20, 2025.

[12] D. Schumacher, E. Nourbakhsh, R. Slavin, and A. Rios, "Prompting underestimates llm capability for time series classification," *arXiv preprint arXiv:2601.03464*, 2026.

[13] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *The Eleventh International Conference on Learning Representations*, 2022.

[14] J. Narwariya, P. Malhotra, L. Vig, G. Shroff, and T. Vishnu, "Meta-learning for few-shot time series classification," in *Proceedings of the 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 28–36.

[15] Y. Jiang, Z. Pan, X. Zhang, S. Garg, A. Schneider, Y. Nevmyvaka, and D. Song, "Empowering time series analysis with large language models: A survey," *arXiv preprint arXiv:2402.03182*, 2024.

[16] J. Ye, W. Zhang, K. Yi, Y. Yu, Z. Li, J. Li, and F. Tsung, "A survey of time series foundation models: Generalizing time series representation with large language model," *arXiv e-prints*, pp. arXiv–2405, 2024.

[17] F. Shi, X. Yin, K. Wang, W. Tu, Q. Sun, and H. Ning, "Large language models for time series analysis: Techniques, applications, and challenges," *arXiv preprint arXiv:2506.11040*, 2025.

[18] C. Sun, H. Li, Y. Li, and S. Hong, "Test: Text prototype aligned embedding to activate llm's ability for time series," *arXiv preprint arXiv:2308.08241*, 2023.

[19] Z. Pan, Y. Jiang, S. Garg, A. Schneider, Y. Nevmyvaka, and D. Song, "S^2IP-LLM: Semantic space informed prompt learning with LLM for time series forecasting," in *International Conference on Machine Learning*. PMLR, 2024, pp. 39 135–39 153.

[20] P. Langer, T. Kaar, M. Rosenblattl, M. A. Xu, W. Chow, M. Maritsch, R. Jakob, N. Wang, J. Liu, A. Verma *et al.*, "Opentslm: Time-series language models for reasoning over multivariate medical text-and time-series data," *arXiv preprint arXiv:2510.02410*, 2025.

[21] Y. Chen, Z. Li, C. Yang, X. Wang, and G. Xu, "Large language models are few-shot multivariate time series classifiers," *Data Mining and Knowledge Discovery*, vol. 39, no. 5, p. 66, 2025.

[22] M. Tan, M. A. Merrill, V. Gupta, T. Althoff, and T. Hartvigsen, "Are language models actually useful for time series forecasting?" *Advances in Neural Information Processing Systems*, vol. 37, pp. 60 162–60 191, 2024.

[23] S. Roschmann, Q. Bouniot, V. Feofanov, I. Redko, and Z. Akata, "Time series representations for classification lie hidden in pretrained vision transformers," *arXiv preprint arXiv:2506.08641*, 2025.

[24] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.

[25] C. Hokamp and Q. Liu, "Lexically constrained decoding for sequence generation using grid beam search," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017, pp. 1535–1546.

[26] T. Koo, F. Liu, and L. He, "Automata-based constraints for language model decoding," *arXiv preprint arXiv:2407.08103*, 2024.

BIOGRAPHY SECTION

If you have an EPS/PDF photo (graphicx package needed), extra braces are needed around the contents of the optional argument to biography to prevent the LaTeX parser from getting confused when it sees the complicated `\includegraphics` command within an optional argument. (You can create your own custom macro containing the `\includegraphics` command to make things simpler here.)

If you include a photo:

Michael Shell Use `\begin{IEEEbiography}` and then for the 1st argument use `\includegraphics` to declare and link the author photo. Use the author name as the 3rd argument followed by the biography text.

If you will not include a photo:

John Doe Use `\begin{IEEEbiographynophoto}` and the author name as the argument followed by the biography text.